Introduction

In this laboratory assignment, you will explore the fundamentals of graphics programming at a low level, focusing on direct interaction with the graphics hardware. By using the VESA BIOS Extensions (VBE) standard, you will learn how to configure graphics modes and access the linear frame buffer to draw graphical primitives. This lab builds upon previous work with text-based output and introduces pixel-based graphics, highlighting key concepts such as video mode switching, memory mapping, and frame buffer manipulation.

Modern graphical applications rely on efficient and hardware-independent access to graphics devices. At a low level, this interaction is made possible through standardized interfaces that abstract the underlying hardware while allowing control over video modes and memory. One such interface is the VESA BIOS Extensions (VBE) standard, which extends the original VGA specification by providing support for higher resolutions, increased color depth, and linear frame buffer access.

In this laboratory, you will explore the organization of video memory and the principles behind pixel-based graphics operations, such as switching to the graphic mode and writing to the frame buffer. Understanding these concepts is essential for developing graphical applications, and they form the basis for more advanced techniques used in interactive systems, games, and graphical user interfaces. The knowledge acquired in this lab will therefore be fundamental to the graphical components of the LCOM final project.

Lab5: Core objectives

- Learn how to use the PC’s video card in graphics mode, using the BIOS/VBE interface for its configuration
- Learn the principles of computer graphics animation
- Introduce relevant techniques for the LCOM final project.

Plan

In the first class, you will learn about the several graphical modes and how they can be configured. Additionally, you will be able to start simple graphic primitives on the screen. Therefore, you will be requested to implement two functions. The first function should be able to switch from the text mode you are familiar with to the specified graphical mode. The second function must not only perform this mode switch, but also draw a rectangle in a specified position of the screen by directly accessing and writing to the video memory.

In the second class, you will learn how to represent images as C data structures using the XPM format, and how to render them on the screen. The final function you will implement before starting the project will be responsible for drawing the provided XPM on the specified screen position. Finally, you will be introduced to some graphical techniques that will be useful for the development of your LCOM final project.

We advise you to read everything carefully, refrain from a blind use of AI, and use the support of the teaching assistants whenever you feel stuck. Good luck and have fun!

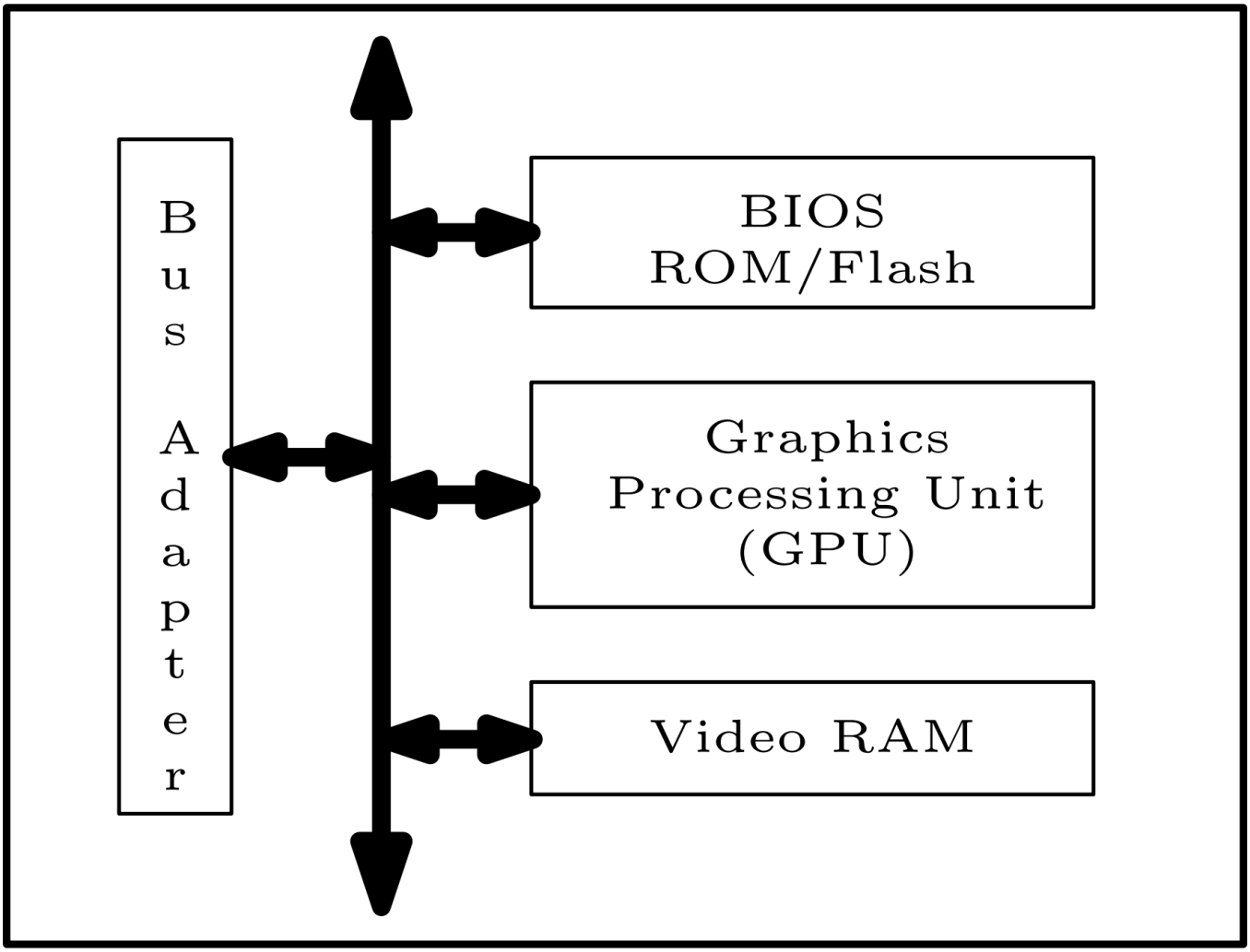
References

Here you have some relevant files and documentation for the The PC's Video Card in Graphics Mode lab:

- lab5.c file that contains the test function stubs that you will develop
- Doxygen documentation
- VESA BIOS EXTENSION (VBE), Version 2.0

The PC’s Video Card

A graphics card, also known as a video card or graphics adapter, is a specialized hardware component responsible for generating and displaying visual output on a computer system. It offloads graphical processing tasks from the central processing unit (CPU), allowing the system to render images, video and animations more efficiently.



Video Card

A video card (graphics adapter) is a specialized expansion card responsible for generating and displaying visual output. It consists of several key components that communicate through internal buses and the system bus.

- **Bus Adapter -** Provides the physical and logical connection between the video card and the motherboard, enabling data transfers between the CPU, system memory, and the video card.

- **Graphics Processing Unit (GPU) -** Main processing component of the video card capable of performing 2D and 3D rendering algorithms for images, videos, and 3D graphics, accelerating graphics applications.
- **Video RAM (VRAM) -** Dedicated memory used exclusively by the GPU to store textures, frame buffers, color data, and depth information. Typically provides higher bandwidth than system RAM, enabling fast access by the GPU.
- **BIOS ROM / Flash (BIOS) -** Firmware stored in ROM or flash memory on the video card responsible for the GPU initialization during system startup while ensuring compatibility between the video card, motherboard, and operating system. Superseded by the UEFI (Unified Extensible Firmware Interface) in modern systems, which usually come with BIOS emulation for backward compatibility.

> **Important**
>
> The **Video Card** can be configured in two different modes: **Text Mode** and **Graphics Mode**

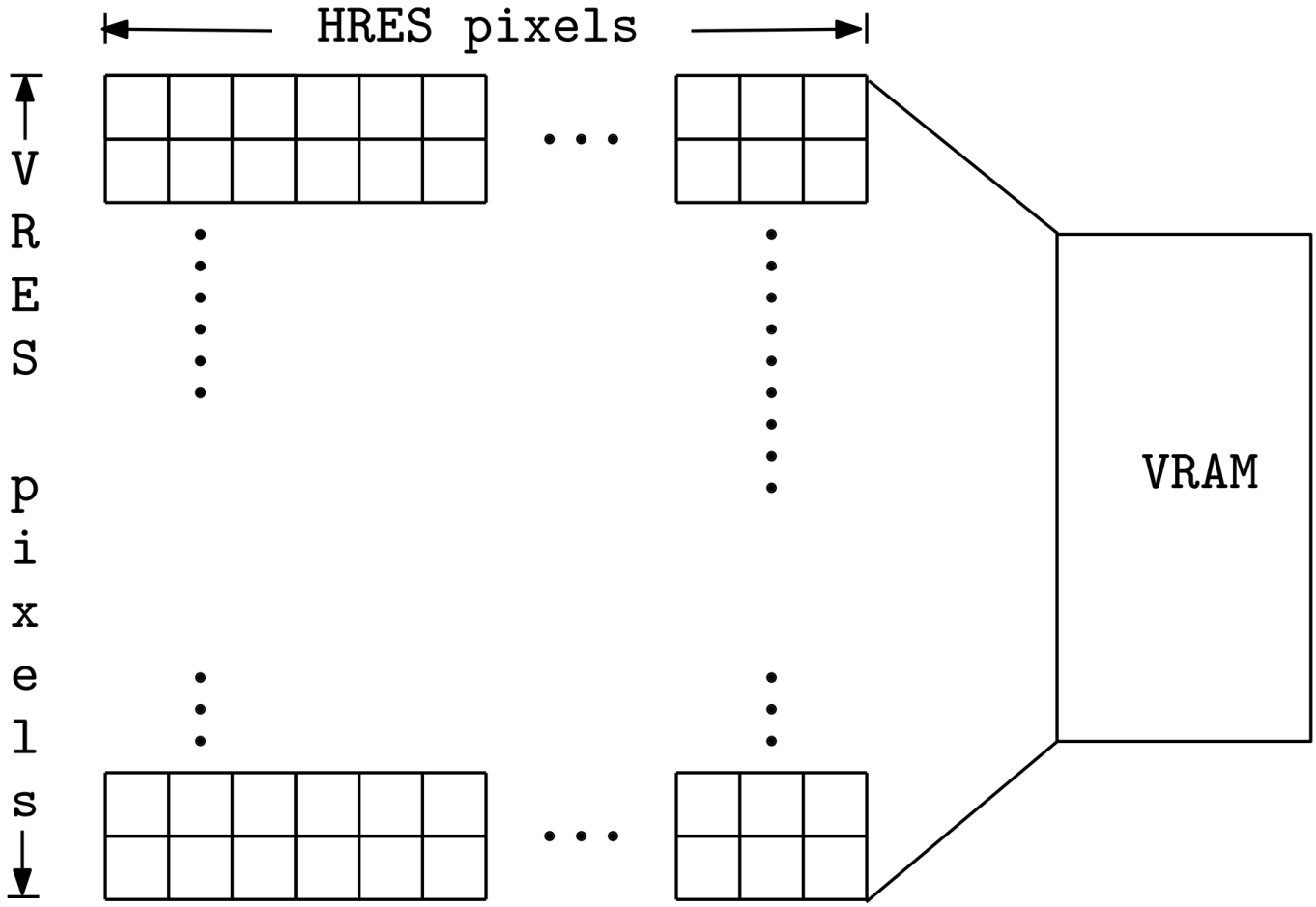
Text Mode

The default MINIX mode is the **Text Mode**, which is mostly used to render text. This is done by modeling the screen as a matrix of characters with a predefined number of rows and columns, e.g., **25x80**, 25x40, 50x80, 25x132.

Row \ Col	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**...**	**80**
0	H	i		W	o	r	l	d	!			
1												
2												
3												
...												
25	m	i	n	i	x	$						

Graphical Modes

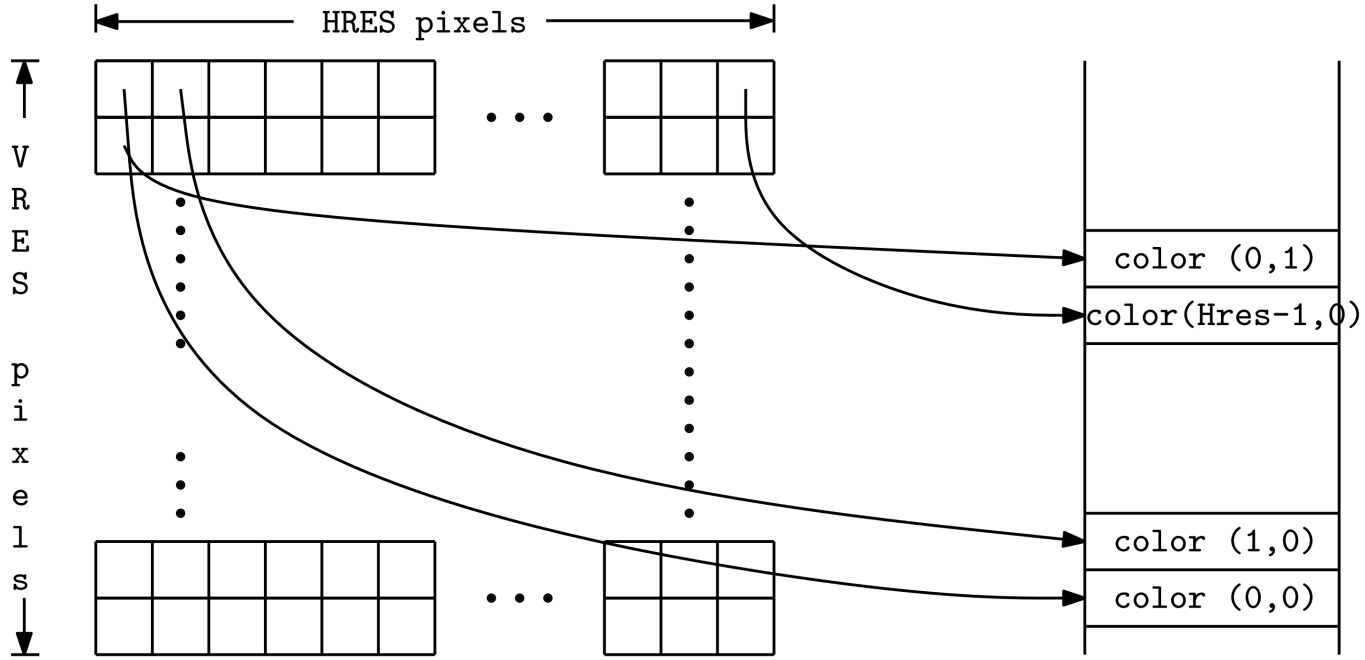
When switching to a **Graphical Mode**, the screen is abstracted as a matrix of **pixels**. The pixels are encoded into the **VRAM**, the dedicated GPU memory responsible for storing each pixel color. The following picture depicts a computer screen where **HRES** and **VRES** is the number of pixels per line and per column, respectively.



Furthermore, the values of the pixels on the screen are stored in **VRAM** respecting the layout enforced by the chosen memory model. In LCOM, we will follow the **Linear/Flat Frame Buffer** Model.

Linear/Flat Frame Buffer Model

In the **Linear/Flat Frame Buffer** model, the screen image is stored in video memory (VRAM) as a continuous, **linear array of pixel data**. The lines of the screen are arranged **sequentially in memory**, starting with the topmost line of the screen and proceeding line by line to the bottommost line. Finally, within each line, pixels are stored from **left to right**, meaning the leftmost pixel appears **first** in memory, until the rightmost pixel, which is stored **last**.



Therefore, to access VRAM and update the Pixel values, it is only required to know:

- The **base address** of **frame buffer**, which points to the **first pixel**
- The **coordinates** of the pixel
- The **color encoding** specifications

> **Note**
>
> Later, you will learn how to obtain the **frame buffer** address and investigate the different **Color Encodings**.

Next Steps

Now that you know how the screen can be abstracted to an array of pixels, we will dive into the VBE standard, which provides an standard interface between the system software and the graphics hardware.

VESA BIOS Extensions Standard

In the late 1980s, a large number of video card manufacturers were offering graphics cards with higher resolutions than those supported by the VGA standard. To help application programmers develop portable video-based software, the Video Electronics Standards Association (VESA) published the **VESA BIOS Extensions (VBE) standard** in 1989. VBE provided a **uniform interface** to access **higher-resolution graphics** and more colors **without needing custom drivers** for each card. During the 1990s, the standard was revised several times, with the last major version, VBE 3.0, released in 1998. By then, other graphics standards such as Microsoft's DirectX and SGI's OpenGL had emerged, which reduced the need for further VBE updates.

The **VBE** defines a set of functions related to the operation of the video card, for software applications that **require access** to the graphics card and its functions.

PC BIOS

The **Basic Input-Output System** (BIOS) is a firmware interface for accessing PC HW resources and it is used mostly to load the OS when a PC starts. In this laboratory, however, we will use the BIOS to access the video card and its functions. The BIOS provides a set of **services** that allow software applications to interact with the hardware components of the PC, including the video card. These services are accessed via software interrupts, which are a mechanism for invoking specific functions provided by the BIOS.

> **Important**
>
> Therefore, the **PC BIOS** plays a **crucial role** acting as a **bridge** between software and the graphics card.

BIOS Services

The BIOS services that provide access to PC HW resources can be accessed via a SW interrupt instruction: **INT XX**

- The **XX** is 8 bit and specifies the service
- Any arguments required are passed via processor registers

BIOS Standard Services

Interrupt Vector (**XX**)	Service
10h	Video Card
11h	PC configuration
12h	Memory configuration
16h	Keyboard

BIOS Call

Originally, the interface to these functions used only INT 0x10, i.e. the interrupt number used by the BIOS' video services, and had to be executed with the CPU in real mode.

```
; set video mode

MOV AH, 0 ; function (set video mode)
MOV AL, 3 ; text, 25 lines X 80 columns, 16 colors
INT 10h
```

Modern operating systems, including Minix, however, for reasons of memory safety and security, run in protected mode. To access the BIOS functions, the operating system must temporarily switch to real mode, execute the BIOS call, and then switch back to protected mode. This is typically done using a special API provided by the operating system, such as `SYS_INT86` in Minix.

VBE Functions

The following table presents the different VBE functions and required paramenters:

Function	AH	AL	Notes
Set VBE mode	0x4F	0x02	Mode is passed in register **BX**, which should have **bit 14 set**
Return VBE Mode Info	0x4F	0x01	Mode is passed on **CX** and must provide address for returned info in registers ES:DI
Return VBE Controller Information	0x4F	0x00	Must provide address for returned info in registers ES:DI

Function	AH	AL	Notes
BIOS Set Video Mode	0x00	<video_mode>	Not a VBE function, but a BIOS function: set AL to 0x03 to return to Minix's default **text mode**

The AH and AL registers are, in fact, the **most significant byte** and the **least significant byte** of the AX register, respectively. Therefore, the VBE functions can also be specified by setting the AX register to the appropriate value, e.g., 0x4F02 for the "Set VBE mode" function. This is an artifact of the x86 architecture, where the same register can be accessed in different ways depending on the required granularity (8-bit, 16-bit, or 32-bit).

Invoking VBE Functions

When invoking a VBE function, the **AH register** must be set to **0x4F**, distinguishing it from standard VGA BIOS functions. The **VBE function** being called is specified in the **AL register**. The use of the other registers depends on the VBE function.

To facilitate the access to these registers, the `<machine/x86.h>` header file includes the `reg_86_t` struct, which can be populated with the required information:

```
reg86_t reg;
memset(&reg, 0, sizeof(reg)); // set reg86_t reg to zeros

reg.intno = 0x10;             // Use INT 0x10
reg.ah = 0x4F;                // set AH=0x4F
reg.al = 0x02;                // specify function in AL
```

To invoke the **VBE function** via **INT 0x10**, you must use `sys_int86`, which temporarily switches from 32-bit protected mode to 16-bit real mode to perform a software interrupt. The `sys_int86()` functions returns **0** in the case of success, or **EFAULT** otherwise, and takes as arguments a pointer of type `reg86_t`.

```
// #include <machine/int86.h> no need to include this line
if (sys_int86(&r) != OK) {
    printf("sys_int86() failed \n");
    return 1;
}
```

> **Note**
>
> Even though the kernel call `SYS_INT86` was dropped in Minix 3.2, it was ported back to Minix 3.4.0rc6 via `libx86emu`, a small library that emulates some key x86 instructions. Therefore, you are allowed to use the `sys_int86` API.

The reg86_t

As the number of bits of the x86 architecture increased, their names changed.

- general-purpose **8-bit** registers were named **A**, **B**, **C**, etc.
- general-purpose **16-bit** registers were named **AX**, **BX**, **CX**, etc.
- general-purpose **32-bit** registers were named **EAX**, **EBX**, **ECX**, etc.

For **compatibility** reasons, the names of **older registers continued to be used** for accessing the **lower-significant bits** of new registers. For example, the **MSB** of the **AX register** could be accessed directly as the **AH register**, and the **LSB** could be accessed directly as the **AL register**.

> **Tip**
>
> Check out the struct reg86_t and the following examples:

```
reg86_t reg;
memset(&reg, 0, sizeof(reg)); // set reg86_t r to zeros

reg.ax= 0x4F03;             // set ax with 16 bit value
reg.al == 0x03;             // true
reg.ah == 0x4F;             // true

reg.ecx = 0xFFFF45ED;       // set the ecx with a 32 bit value
reg.cl == 0xED;             // true
reg.ch == 0x45;             // true
reg.cx == 0x45ED;           // true

reg.vec = 0x02043D74;       // set the ecx with a 32 bit value
reg.off == 0x3D74;          // true
reg.seg == 0x0204;          // true
```

VBE Modes

The VBE provides several video modes by supporting both **indexed** and **direct color** modes across a range of **screen resolutions**, from standard VGA sizes to higher-resolution displays. The following table presents available video modes, highlighting the relationship between display resolution, color model, and color depth, i.e., the number of bits required to represent a color.

Mode	Screen Resolution	Color Model	Bits per pixel (R:G:B)
0x105	1024x768	Indexed	8
0x110	640x480	Direct color	15 (1:5:5:5)

Mode	Screen Resolution	Color Model	Bits per pixel (R:G:B)
0x115	800x600	Direct color	24 (8:8:8)
0x11A	1280x1024	Direct color	16 (5:6:5)
0x14C	1152x864	Direct color	32 (8:8:8:8)

> **Note**
>
> For now, don't worry about the **Color Model** and **Bits per pixel** columns. Later, we will further investigate them.

Next Steps

Now that you know the basics of VBE and its functions, let's switch from the default Text mode to the Graphic mode in Minix: Switching to Graphic Mode

Switching to Graphical Mode

Now that you have studied the VBE, it is time to begin the first task of Lab 5. You shall implement the `video_test_init` function to switch the display from text mode to a pixel-based graphics mode.

> **Tip**
>
> To help you, we have prepared the lab5.c file, which contains the stubs of the main functions to implement in this lab, together with doxygen's documentation of these functions and other useful utilities.

The video_test_init(uint16_t mode, uint8_t delay) function

The purpose of this function is that you learn how to switch the video adapter to the **Graphics mode** specified in its argument, using the **VBE interface**, and then back again to the default text mode.

1. When this function is invoked, your program should **change** to the **Graphics mode** specified in its `mode` argument
2. Wait `delay` seconds
3. Finally, the video adapter should go back to Minix's default **Text mode**

> **Note**
>
> In graphics mode you will not have access to the Minix Terminal, and hence to the shell. Thus, before terminating your program, you should always reset the video controller to operate in text mode.

Switching to Graphic Mode by Invoking the Set VBE Mode (0x02) function

To initialize the controller and set a video mode, you should use function **Set VBE Mode - 0x02** and populate the **BX register** with the desired **mode**. Do not forget to set **Bit 14** of *BX register* in order to use the linear frame buffer model, facilitating the access to video RAM (VRAM).

Furthermore, when invoking the VBE function **Set VBE Mode - 0x02**, the **bit 15** of the **BX register** should be unset, ensuring that the display memory is cleared after switching to the desired graphics mode. This can be achieved using the `memset()` function before populating the `struct reg86_t`.

> **Tip**
>
> - Check out: Invoking VBE Functions
> - Check out: `memset()` by executing `man memset` in MINIX terminal

Returning to Text Mode

To return to Text Mode, you can use the function `vg_exit()`, which is **already provided** by the LCF.

The mode used by Minix in **Text mode** is a standard CGA mode, whose number is **0x03**. To set this mode, `vg_exit()` implementation uses the standard INT 0x10 BIOS interface, namely function **0x00**. Thus, the **AH register** is set to **0x00** and the **AL register** to **0x03**.

```
/* Set default text mode */
int vg_exit() {
  reg86_t r86;

  /* Specify the appropriate register values */

  memset(&r86, 0, sizeof(r86)); /* zero the structure */

  r86.intno = 0x10;             /* BIOS video services */
  r86.ah = 0x00;                /* Set Video Mode - standard BIOS function */
  r86.al = 0x03;                /* 80x25 text mode */

  /* Make the BIOS call */

  if (sys_int86(&r86) != OK) {
    printf("\tvg_exit(): sys_int86() failed \n");
    return 1;
  }
  return 0;
}
```

Running video_test_init() function

After compiling using the `make` tool, you can execute the `video_test_init()` function using the **lcom_run utility** as follows:

```
minix$ lcom_run lab5 "init <mode (hex)> <delay (seconds)>"
```

Testing with LCF

Finally, you should test the `video_test_init()` function using the **LCF testing mode** by adding the string **"-t <test no.>"** at the end of the command line. This function **only accepts 1** as the value of **<test no.>**.

```
minix$ lcom_run lab5 "init <mode (hex)> <delay (seconds)> -t 1"
```

Test **nº 1** is a parameterized test that allows the use of arbitrary (but appropriate) values for both the mode and the delay. Verification is performed by intercepting the relevant function calls and checking both their arguments and the time instants at which they occur.

Next Steps

Well done, you were able to switch from Text mode to a Graphical mode! We will now explore how to access and manipulate video memory to draw on the screen in Accessing and Manipulating Video Memory.

Accessing and Manipulating Video Memory

Despite the fact that you are now capable of switching to graphical mode, the screen remains black. How can we draw on the screen? The answer is by **accessing** and **manipulating** video memory, i.e., the **VRAM**!

In this chapter, you will learn how colors are encoded in video memory (VRAM). Additionally, you will explore how to access and manipulate this memory in order to correctly paint the screen.

Colors Encoding

Nowadays, most electronic display devices use the **RGB color model**, in which a color is obtained by adding the 3 primary colors: **red**, **green**, and **blue**. Therefore, we can represent a color using **triples** containing a given **intensity** per primary color. Depending on the chosen **Graphic Mode**, each primary color intensity can have **different number of bits**.

> **Tip**
>
> If we represent each primary color with **8 bits** following the **RGB format** we get:
>
> - **24 bits** per pixel in format **(8:8:8)**
> - 2^{24} = 16777216 different colors

As you may already noticed when the **VBE Modes** were presented, the colors can follow different **Color Models** and be represented in different **formats**.

Recall the **VBE Modes table**, now focusing in the Color Model and Bits per pixel columns.

Mode	Screen Resolution	Color Model	Bits per pixel (R:G:B)
0x105	1024x768	**Indexed**	8
0x110	640x480	**Direct color**	15 **(1:5:5:5)**
0x115	800x600	**Direct color**	24 **(8:8:8)**
0x11A	1280x1024	**Direct color**	16 **(5:6:5)**
0x14C	1152x864	**Direct color**	32 **(8:8:8:8)**

> **Note**

> In **0x110** mode, colors are represented using **15 bits**. However, since memory is byte-addressable and the PC allocates data in **units of 8 bits** (1 byte), each pixel occupies **2 bytes** (16 bits) in video memory, with one bit remaining **unused**.

Direct Color Model

In VBE Modes using Direct Color mode, the color information is **directly stored in each pixel**. The pixel value is divided into separate fields for **red**, **green**, and **blue** components, each using a **fixed number of bits**. This allows a much larger range of colors to be displayed simultaneously, depending on the number of bits per pixel.

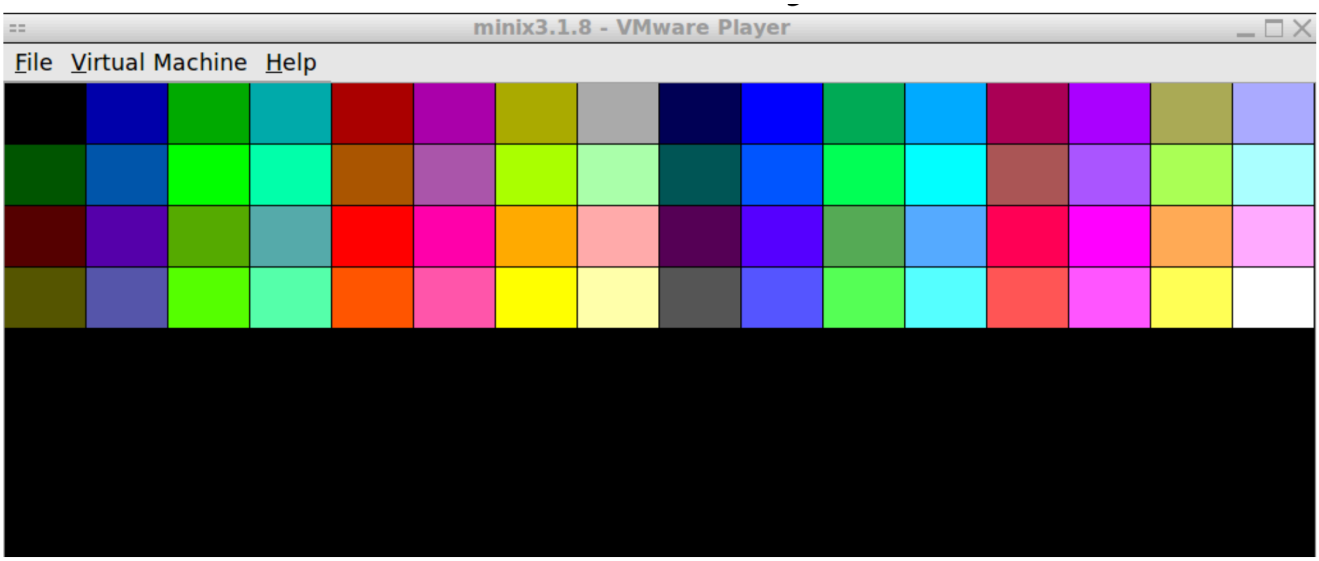
Changing a pixel's color requires **modifying its value in video memory**, i.e., VRAM, but the color representation is more accurate and flexible.

Indexed Color Model

The Indexed Color model takes advantage of a **color palette** to represent colors on the screen. Each pixel stores an **8-bit index** that **points** to a **specific color in a palette table**, rather than storing the color itself. VBE mode **0x105** uses 8 bits per pixel, thus, a pixel can reference up to **256 colors**. The actual RGB values are stored separately in the palette, which allows the same pixel values to display different colors simply by changing the palette.

This mode is memory-efficient but limited in the number of colors that can be shown at once.

> **Note**
>
> In MINIX-LCOM, the default color palette of **VBE Mode 0x105** supports 256 colors, but **only the first 64** are initialized.



Accessing VRAM

As previously explained, the video memory **VRAM** is the dedicated **physical memory region** used exclusively by the **GPU**. Furthermore, in modern operating systems, user-level processes **cannot directly access HW resources**, including physical memory and **VRAM**. To address this, Minix 3 **grants privileged user-level processes** the permissions they require to perform their tasks, providing the necessary kernel calls to access HW resources.

For accessing and controlling the video RAM (VRAM) at the user level, more specifically the frame buffer in VRAM, it is convenient to map the physical memory region corresponding to VRAM into the process's virtual address space. This allows the process to read and write to VRAM as if it were regular memory, enabling direct manipulation of the frame buffer for graphics operations.

1. Obtain VRAM Physical Memory Address

To obtain the physical address of the VRAM, the LCF provides the `vbe_get_mode_info()` function. This function should be called **once** and retrieves information about the current **VBE mode**, including the physical address of the frame buffer, screen dimensions, and color depth.

```
// function already included by <lcom/lcf.h>
int vbe_get_mode_info(uint16_t mode, vbe_mode_info_t *vmi_p);
```

Where:

- `uint16_t mode` : **VBE mode** whose information should be returned, e.g. **0x105**
- `vbe_mode_info_t * vmi_p` : **pointer** to a `struct vbe_mode_info_t` that will be internally initialized
- On success, the function returns **0**

Deep down, `vbe_get_mode_info()` function initializes a `struct vbe_mode_info_t` with the VBE information on the input mode by calling **VBE function 0x01**, which returns VBE Mode information, and copying the `ModeInfoBlock` struct returned by that function.

> **Tip**
>
> - Check out: Definition of struct vbe_mode_info_t

2. Mapping Video RAM in a Process's Address Space

For mapping **physical memory**, Minix requires a process to not only have the necessary permissions to execute that call, but also have the permission to map the desired physical

address range. To **grant** a process the permission to map a given address range of physical memory you should use MINIX 3 kernel call:

```
int sys_privctl(endpoint_t proc_ep, int request, void *p);
```

Where:

- `endpoint_t proc_ep` : specifies the process whose privileges will be affected
- `int request` : permission to be granted: `SYS_PRIV_ADD_MEM`
- `void *p` : memory address range where the permission will be granted
- On success, the function returns **0**

As you already know, what is displayed on the computer screen reflects the contents of the **VRAM frame buffer**. Therefore, to change what is displayed on the screen, we must **modify** the frame buffer. To map the VRAM physical memory region and access the frame buffer, MINIX 3 provides the following kernel call:

```
void *vm_map_phys(endpoint_t who, void *phaddr, size_t len);
```

Where:

- `endpoint_t who` : identifies the process that will receive the mapping of the physical memory into its address space; set to `SELF` to specify the calling process
- `void *phaddr` : starting physical address of the memory region to be mapped; in this case, the physical address of the VRAM frame buffer obtained from `vbe_get_mode_info()`
- `size_t len` : size, in bytes, of the physical memory region to map
- Returns the **virtual address** (of the first physical memory position) on which the physical address range was mapped

After executing `vm_map_phys()` function, the user-level process can use the **returned virtual address** to access the contents in the mapped physical address range. Therefore, you are now able to **modify** the **VRAM frame buffer** and draw on the screen.

Implementation Notes

To easen the development process of this lab, you can implement functions that are not generic on the address on which VRAM is mapped, or the horizontal and vertical screen resolution, or the color model. Take advantage of **static global variables** in the **source code file where VRAM's content is modified**, and program against a fixed model, as shown in the following code snippet:

```
static char *video_mem;          /* Virtual address to which VRAM is mapped */

/* Other variables that might be used */
static vbe_mode_info_t vmi;      /* VBE mode info */
static unsigned bytes_per_pixel; /* Number of VRAM bytes per pixel */
```

Despite the fact that `vm_map_phys()` function returns `void *`, we may declare the pointer to the VRAM as a `char *`. This choice allows the pointer to address **individual bytes**, as a `char` is 1 byte and pointer arithmetic in C offsets by the type width. This is essential for correctly computing memory offsets and to be compatible with all **VBE Modes**. Using pointers such as `uint16_t *` or `uint32_t *` would implicitly assume a fixed pixel size, making the code dependent on specific color depths.

> **Tip**
>
> Although the use of global variables should be **avoided**, there are two reasons why they are **acceptable**:
>
> 1. These are **static** global variables, thus, their scope is **limited to the translation unit where they are defined**
>   - **static global variables** are not visible in other files
> 2. We are organizing our code similarly to object-oriented programming
>   - These variables are mainly used to keep state information
>   - Implement **get() methods** to access them outside the scope

Summary Code Snippet

With the previous presented functions, you are now able to access the **VRAM**. The following code snippet summarizes the process of accessing the **VRAM frame buffer** following the previous presented steps. You may use, adapt, and complete the snippet according to your needs.

```
/* Static global variables */
static char *video_mem;          /* frame-buffer VM address */

/* Other variables that might be used */
static vbe_mode_info_t vmi;      /* VBE mode info */
static unsigned bytes_per_pixel; /* Number of VRAM bytes per pixel */

// TIP: You can add other "static global variables" as you see fit.

[...]

int r;
struct minix_mem_range mr;
unsigned int vram_base;          /* VRAM's physical addresss */
unsigned int vram_size;          /* VRAM's size, but you can use the frame-
buffer size */

/*
To be implemented:
1. Use vbe_get_mode_info() to initialize vmi
2. Set vram_base and vram_size according to VBE mode information
3. Set other relevant static global variables
*/

/* Grant memory mapping permissions */
mr.mr_base = (phys_bytes) vram_base;
mr.mr_limit = mr.mr_base + vram_size;

if (OK != (r = sys_privctl(SELF, SYS_PRIV_ADD_MEM, &mr)))
   panic("sys_privctl (ADD_MEM) failed: %d\n", r);

/* Map memory */
video_mem = vm_map_phys(SELF, (void *)mr.mr_base, vram_size);

if(video_mem == MAP_FAILED)
   panic("couldn't map video memory");
```

Next Steps

Now that you know how pixel colors are represented and can access VRAM, it's time to put that knowledge into practice and start drawing: Draw Rectangles on Graphic Mode

Draw Rectangles in Graphical Mode

These tasks will require you to implement a function that changes the color of screen pixels. The end goal is to draw a solid-colored rectangle at a given position on the screen.

The video_test_rectangle(...) function

The `video_test_rectangle()` function sets the desired **graphics video mode** and draws a **filled rectangle** on the screen. It receives the rectangle's **position**, **dimensions**, and **color** as parameters, writes the corresponding pixels directly to video memory after mapping it to the process adress space. Finally, this function must return to the **Text mode** and terminate when the breakcode of the ESC key, **0x81**, is received.

The implementation should follow the next steps:

1. **Map the video memory** to the process' address space
2. **Change the video mode** to the mode specified
3. Draw a rectangle by **writing in VRAM**
4. Reset **Video mode** to Minix's default **Text mode** and return upon receiving the **breakcode of the ESC key**

> ⚠ Warning
>
> Steps 1 and 2 order can swapped. However, the LCF tests are **not flexible**, requiring **step 1 to be performed before step 2**. Otherwise, LCF tests will fail with the message: `Test FAILED: one or more missing/unexpected function calls!`

The `video_test_rectangle()` function declaration is the following:

```
int video_test_rectangle(uint16_t mode, uint16_t x, uint16_t y,
                         uint16_t width, uint16_t height, uint32_t color);
```

Where:

- `uint16_t mode` : **VBE mode** whose information should be returned, e.g. **0x105**
- `uint16_t x` : rectangle top-left **x coordinate**, in pixels
- `uint16_t y` : rectangle top-left **y coordinate**, in pixels
- `uint16_t width` : rectangle width, in pixels
- `uint16_t height` : rectangle height, in pixels
- `uint32_t color` : color to fill the rectangle, encoded as determined by the first argument, mode.

To facilitate the development process, we recommend the implementation of the following helper functions:

```
int vg_draw_hline(uint16_t x, uint16_t y, uint16_t len, uint32_t color);
int vg_draw_pixel(uint16_t x, uint16_t y, uint32_t color);
```

> **Tip**
>
> Check out: `vg_draw_hline()` documentation

Pointer Arithmetic

As you already know, a C pointer is a data type whose value is a memory address, which allows you to do the following basic operations:

```
int a[5];       /* array of 5 integers */
int * p;        /* pointer to an integer */

p = a;          /* set pointer p to the address of the first element in array a
*/
p = &(a[4]);    /* set pointer p to the address of the last element in array a
*/
```

> **Note**
>
> Keep in mind that `&(a[4])` is **sintax sugar** for the pointer arithmetic expression `a + 4`. In this case, though, the sugar makes things worse—like frosting a bad cake.

When dealing with C arrays, such as the **VRAM frame buffer**, keep in mind that all the values are stored in **contiguous memory locations**. As such, taking advantage of pointer arithmetic can help to easily access a **specific position** in the array or to iterate over an array, as in the following examples:

```
int a[5];       /* array of 5 integers */
int * p;        /* pointer to a integer */


/* Get value at the second position of array a */
p = a;
int a_idx_2 = *(p + 2);

/* Iterate over the array */
for (int i = 0, p = a; i < 5 ; i++, p++) { int a_idx = *p; }

/* Same example but without variable i */
for (p = a; p - a < 5 ; p++) { int a_idx = *p; }
```

> **Tip**
>
> In C, pointer arithmetic is relative to the width of the type it a pointer points to. When you increment a pointer (`ptr++`), the generated machine code **does not simply add 1** to the memory address; it **increments the size of the data type** it points to. Be aware that, in most cases, the pointer type should be **equal** to the type of each element in the array.

Changing Pixel Values in VRAM

After reviewing the basics of Pointer Arithmetic, you are now able to obtain the address of each pixel or iterate over the mapped VRAM video buffer using the returned VBE mode information from calling the `vbe_get_mode_info()` function and the virtual address returned from `vm_map_phys()` .

> **Tip**
>
> - Check out: Obtain VRAM Physical Memory Address
> - Check out: Mapping Video RAM in a Process's Address Space

Changing the pixel value requires you to **write the desired color in the VRAM address corresponding to that pixel**. Thus, depending on the VBE mode, you should write the **correct number of bits** in that specific address. For example, if the desired **VBE mode 0x105** is chosen, each pixel takes only **1 byte**, and in case of **VBE mode 0x115**, you must write **24 bits**.

Since the `uint32_t color` argument has 4 bytes and there are modes that use **less memory** to represent a color, you should always consider the **least significant bytes** present in the `color` argument. Finally, you have to be careful when using **VBE mode 0x110** since it uses **15 bits** to represent the color and not the usual 16 bits in every 2 bytes.

Running video_test_rectangle() function

After compiling using the `make` tool, you can execute the `video_test_rectangle()` function using the **lcom_run utility** as follows:

```
minix$ lcom_run lab5 "rectangle <mode (hex)> <x> <y> <width> <height> <color
(hex)>"
```

Testing with LCF

Finally, you should test the `video_test_rectangle()` function using the **LCF testing mode** by adding the string **"-t <test no.>"** at the end of the command line. This function **only accepts 1** as the value of **<test no.>**.

```
minix$ lcom_run lab5 "rectangle <mode (hex)> <x> <y> <width> <height> <color
(hex)> -t 1"
```

Test **nº 1** is a parameterized test that allows the use of arbitrary (but appropriate) values for all arguments. Verification is performed by analysing the contents of the frame-buffer upon calling vg_exit(). Furthermore, the LCF injects the makecode and breakcode of the ESC key as soon as your code invokes driver_receive() after subscribing keyboard interrupts.

Pixmap and XPMs

As you already implemented in the previous task, the drawing process involves writing color values directly to video memory **VRAM** so that each pixel on the screen displays the intended color. Therefore, by **carefully controlling how pixels are placed and colored**, it is possible to render anything from **simple shapes** like a rectangle, to **complex images**.

Drawing images on the screen is a fundamental task in computer graphics, as it allows programs to visually represent information and create interactive interfaces. Therfore, let's learn how images can be represented, facilitating the image drawing process.

Pixmap

A **Pixmap**, short for "pixel map", is a general concept that represents an image as an matrix/array of pixels, where each pixel directly stores its color value. Basically, Pixmaps are maps of screen coordinates to color values, making them simple and efficient to draw since rendering them usually consists of **copying pixel data** directly into the video memory **VRAM**.

XPM

An **XPM**, short for "X PixMap", is a specific file format and textual representation of a pixmap, avoiding images to be stored as raw binary pixel data. This enables the encoding of **Pixmaps** in human-readable ASCII characters and a color table that maps characters to colors.

Therefore, an **XPM** for a given **Pixmap** can be stored, for example, as a text file or as an array in a C source file, making XPM files easy to embed in source code.

Storing XPMs as C arrays: Legacy Format

During the development of this lab, you will use XPMs in the **XPM Legacy Format** together with the Indexed Color Model (**VBE Mode 0x105**). Following this format, each **pixel** is represented by a **character**, which corresponds to one of the **indexed colors**. In the following example, the character '.' corresponds to the indexed color 0, and the character 'x' corresponds to the indexed color 2.

```
static xpm_row_t const pic1[] = {
    "32 13 4",      /* number of columns and rows, in pixels, and colors */
    ". 0",          /* '.' denotes color value 0 */
    "x 2",          /* 'x' denotes color value 2 */
    "o 14",         /* .. and so on */
    "+ 4",
    "................................", /* pixels */
    "..............xxx...............",
    "............xxxxxxx.............",
    "........xxxxxx+xxxxxx..........",
    "......xxxxxxx+++++xxxxxxx.......",
    "....xxxxxxx++++++++xxxxxxx.....",
    "....xxxxxxx++++++++xxxxxxx.....",
    "......xxxxxxx+++++xxxxxxx.......",
    "........xxxxxx+xxxxxx..........",
    ".........ooxxxxxxxoo...........",
    ".......ooo...........ooo........",
    ".....ooo...............ooo......",
    "...ooo...................ooo...."
};
```

Storing XPMs as C arrays: XPM3 Format

Since the previous XPM format only works when using indexed colors, you may want to use the **XPM3** format in the project. This format allows colors to be defined in the RGB format (**#XXYYZZ**). The following snippet of C code presents a portion of a XPM, which can be included and used in other C files of your program.

```
xpm_row_t const minix3_xpm[] = {
    "196 196 950 2",    /* number of columns and rows, number of colors, and
bytes per pixel */
    " c None",          /* 'c' denotes the type color */
    ". c #C1C1C1",    /* 'm' denotes the type monochrome */
    "+ c #323232",    /* 'g' denotes the type grayscale */
    "@ c #090909",    /* 's' denotes the type symbolic */
    "# c #010101",
    "$ c #161616",
    "% c #9C9C9C",
    [...],
    "
. + @ # $ %                                                 ",
    "                                                             & * = - ;
> , ' )       ! ~ { ] ^ / ( _ : <                                          ",
    "                                                        [ } | | | | | 1 2 3 4 5
6 7 8 9
0 a / | | b c d e f ] g h @ g i i i j k
",
    "                                                   ~ | | | < l : m | | | | |
n o # | | | p q r 0
s l | | | k | | t u v w ] < x 5 i i i 5 y z
",
    "                                                 A B | | C i i D E F G H m |
| | | | | | | | | | @ I J K
L m | M ] N O P n Q ] { R S T i i i i i U p v V W
",
    "                                                 X | | Y i i i i i i i i i Z
`  .c ..+.@.T #.$.%.| | | # Y [ &.
*.    =.-.;.m 1 >.,.i i i '.o b Y ).0 i i i i i i i t | p | !.
",
    "                                               ~.# | 1 U i i i i i i i i i i
i i i {.].^./.i i i 9 (.m | | | | Y F
[ ^ # _.[ ] { :.} <.i i i i i i i %.h [.i i i i i i i i }.n | | | |  .
",
    [...]
}
```

> **Tip**
>
> - GIMP allows you to **export images** in XPM3 format, where each pixel/color has 3 bytes
> - Check out: XPM3 format

Reading a Pixmap from a XPM file

To facilitate the conversion of a **XPM** into a **Pixmap**, LCF provides the `xpm_load()` function. This function assumes that the **XPM** either uses **one char per color** and one byte per color (**must be used with the VBE mdoe 0x105**) or **3 bytes per color**, with no restriction on the number of bits per color (format generated by **GIMP**).

```
// function already included by <lcom/lcf.h>
uint8_t *xpm_load(xpm_map_t xmap, enum xpm_image_type type, xpm_image_t *img);
```

Where:

- `xpm_map_t xmap` : struct containing the content of the **XPM** file
- `enum xpm_image_type type` : Supported **output formats**, e.g., XPM_INDEXED, XPM_8_8_8
- `xpm_image_t *img` : Pointer to the pixmap and its metadata
- Returns the address of the **allocated memory** for the **Pixmap** as two-dimensinal `uint8_t` array

> **Tip**
>
> - Check out: `struct xpm_map_t` and `enum xpm_image_type` documentation

Next Steps

Now that you are aware on how images are represented as XPMs, let's implement a function to draw them on the screen: Drawing XPMs

Drawing XPMs

Finally you have reached the last task of Lab 5. The purpose of the function to be implemented is drawing a pixmap provided as an XPM image at a predetermined position on the screen, according to the given arguments.

The video_test_xpm(xpm_map_t xpm, uint16_t x, uint16_t y) function

This function should switch the system to **VBE mode 0x105** and display the pixmap obtained from the given XPM at the specified screen coordinates, `uint16_t x` and `uint16_t y`, corresponding to the pixmap's upper-left corner. When the user releases the **ESC key** (breakcode **0x81**), the function must restore MINIX's default text mode and return.

You can use the `load_xpm()` function provided by the LCF to convert an XPM image into a pixmap ready to be drawn on the screen.

> **Tip**
>
> Check out: Reading a Pixmap from a XPM file

The **XPM** passed as the first argument to video_test_xpm(), `xpm_map_t xpm`, is selected from the set of **XPM images** defined in `pixmap.h` file. The specific XPM used depends on the **second argument** (the one following the "xpm" string) of the **lab5** MINIX service, as shown in the table below.

XPM index: Second argument of lab 5	XPM in the `pixmap.h` file
0	pic1
1	pic2
2	pic3
3	cross
4	penguin

Running video_test_xpm() function

After compiling using the `make` tool, you can execute the `video_test_xpm()` function using the **lcom_run utility** as follows:

```
minix$ lcom_run lab5 "xpm <xpm (index)> <x> <y>"
```

Testing with LCF

Finally, you should test the `video_test_xpm()` function using the **LCF testing mode** by adding the string **"-t <test no.>"** at the end of the command line. This function **only accepts 1** as the value of **<test no.>**.

```
minix$ lcom_run lab5 "xpm <xpm (index)> <x> <y> -t 1"
```

Test **nº 1** is a parameterized test that allows the use of arbitrary (but appropriate) values for all arguments. Verification is performed by analysing the contents of the frame-buffer upon calling vg_exit(). Furthermore, the LCF injects the makecode and breakcode of the ESC key as soon as your code invokes driver_receive() after subscribing keyboard interrupts.

Reactive Graphical Applications

While drawing a fixed set of pixels in a given position sufficed for the purpose of this laboratory guide, you might need to exercise more advanced Computer Graphics techniques in the final project. As your application gathers data from the user, e.g. arrow keypresses, it will need to *react* accordingly, performing some logic that will materialize into a new state, to be drawn in the screen. These architectural challenges should already be familiar to you, from *Software Design and Testing Laboratory*, but solving them at a lower level requires a different set of techniques.

Beyong Simple Graphics

One of the first tasks in your project will likely be getting an entity to move. You first load an asset into memory, parsing a XPM to an array of colors, whose format depends on the set graphics mode:

```
#include "../assets/player.xpm"
xpm_image_t img;
uint8_t* pixmap = xpm_load(xpm, XPM_INDEXED, &img);
```

It is very handy to directly *paste* the XPM character array in your source code via the usage of an `include` directive. In less C-friendly formats, say a JPEG, you'd have to perform some file I/O to read their byte contents, and properly convert it to an array of pixels to transfer to the VRAM, which is not a trivial tasks since there is some compression involved in those alternative formats.

Then, drawing the loaded XPM across some lines should be straightforward...

```
for (uint16_t x = 0; x <= 800; x += 200) {
    vg_draw_pixmap(pix_map, img, x, 200);
    sleep(1); // you'll probably want to pause in a different way...
}
```

The first obvious lacking thing is clearing the previous positions. You might do so by zeroing the memory section occupied by the XPM as drawn in the previous iteration, but, unfortunately, that is not a scalable approach.

The Painter's Algorithm

In 3D Computer Graphics, where you have a set of polygons with X, Y and Z dimensions set in a virtual world, the Painter's Algorithm is traditionally used to determine the order in

which the polygons should be drawn, typically from the farthest to the nearest relative to the viewer. In simpler 2.5D applications, you'll probably be able to determine it before-hand:

- a background
- farthest entities (the enemies?)
- nearest entities (the hero?)
- the cursor
- ...

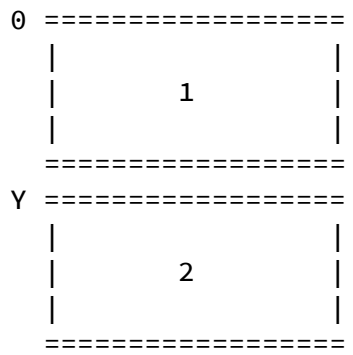
When it is time to refresh the visual state – which should be done at a fixed rate, independently from the logic updates, to avoid the risk of lagging due to excessive VRAM memory writes – copy the assets to memory in order. When the Graphics Card performs its next vertical refresh, it will ideally read from VRAM only after all updates have been written. Otherwise, you will experience screen flickering.

Preventing Screen Flickering

Screen Flickering, sometimes also referred to as *screen tearing*, is a visual artifact characterized by the showing of information from multiple frames (loosely, *drawn app states*), in a single image. Since writing your assets to memory is not an atomic operation – to put it simply, you will be calling several functions for each drawn entity – you might end up writing to graphics memory during a screen refresh, degrading the user experience.

A rather trivial, but still effective solution is to update your drawing routines to write into a separate buffer, allocated in the process virtual memory, instead of directly into the VRAM-mapped one. Let's call it the *hidden buffer*. Then, as a last step, perform a single `memcpy` from that buffer into the VRAM: the time performing I/O in the graphical memory will be less significant, reducing the probability of it happening during a screen refresh.

A more sophisticated solution using the VBE interface is resorting to the use of the 07h Function (Get/Set Display Start). In this approach, you'll be requesting the OS 2 times your resolution of memory to map from the start of the physical base of graphics memory. This is fine: remember your graphics card (here, actually, the emulated one) has much more available memory than the amount required to store all screen's pixels. Take a look at the visual diagram.



Buffer 1 starts at line 0 and 2 at line `Y` where `Y` is the vertical resolution. At a certain point in time, the application writes to a `hidden_buffer` given by `(cur_buffer + 1) % 2`, where `cur_buffer` is the index of the buffer set in the previous call of the VBE 07h function. Each VBE call will switch the buffer to display. This, semantically, guarantees that you'll always write into a non-displayed buffer, but avoids the redundant write operations in the process memory.

This is typically called *Page Flipping*.

Animated Sprites

Application entites will likely need to turn sides, walk or generically need to switch the pixmap that represents them at a certain point in time. You should abstract away this behavior by a data structure in C:

```
struct sprite_t {
    uint8_t* pixmap;
};

struct animated_sprite_t {
    uint8_t no_pixamps;
    uint8_t cur_pixamp;
    sprite_t pixmaps[8];
    uint16_t x;
    uint16_t y;
}
```

The `pixmaps` array is your set of possible entity representations. At a given pace, you may switch the pixmap currently shown to represent movement.

A character with several sprites

Detecting Collisions

A simple collision detection function might look like this:

```
bool collide(sprite_t* a, sprite_t* b) {
    return a->x < b->x + b->width &&
           a->x + a->width > b->x &&
           a->y < b->y + b->height &&
           a->y + a->height > b->y;
}
```

However, this fails to capture the fact that some pixels in the sprite might be transparent, and thus not actually colliding with the other entity. A more accurate approach would be to check for each pixel in the sprite if it is transparent or not, and if it is not, check if it collides with the other entity. This might be too slow; you may think of some optimizations, such as only checking for collisions if the bounding boxes of the entities collide, or using a spatial partitioning technique to reduce the number of collision checks.

Drawing Perspective

In a 3D world, where your assets are defined by a set of points that form planes and polygons, the techniques required to determine the visible pixel color at a given screen

position typically involve some form of *Ray Casting*. This is outside the scope of this course unit.

Classic Doom with Ray Casting.

Dealing with State

While in C one cannot directly apply the State Pattern as one would do in an object-oriented language, it is still straightforward to represent your application as a bunch of states, which may be simple functions, or more complex data structures with function pointers. The main idea is to have a main loop that calls the current state function, which will perform some logic and then switch to another state if needed.

```
void main_loop() {
    load_assets();
    state_t cur_state = STATE_MENU;
    for (;;) {
        driver_receive(...);
        // You might need to hook state updates to other devices...
        if ((msg & BIT(timer_pos)) && (++counter % FPS == 0)) {
            switch (cur_state) {
                case STATE_MENU:
                    cur_state = menu_state(...);
                    break;
                case STATE_GAME:
                    cur_state = game_state(...);
                    break;
                case STATE_PAUSE:
                    cur_state = pause_state(...);
                    break;
            }
        }
    }
}
```

Assuming that each state function will return the next state to be executed, this is a simple way to manage the application state. You can also use function pointers to avoid the switch statement, but that is a matter of preference. The main idea is to have a clear separation of concerns between the different states of your application, and to have a main loop that manages the state transitions as *handlers* of device interrupt notifications. The device drivers should not care about the application state, and the application state should not care about the device drivers. This should come naturally as you statically link your application against the low-level drivers you have implemented in the previous labs, and stick to their exposed interfaces.

What's next

Congratulations on completing all the `labs`! Now that you are capable of working with different I/O devices, it's time to focus on your project. Keep up the great work, and have fun!